

prog_datasci_4_numpy

September 24, 2022

1 Fonaments de Programació

1.1 Unitat 4: Llibreries científiques en Python - NumPy

1.1.1 Instruccions d'ús

Aquest document és un *notebook* interactiu que intercala explicacions més aviat teòriques de conceptes de programació amb fragments de codi executables. Per aprofitar els avantatges que aporta aquest format, us recomanem que, en primer lloc, llegiu les explicacions i el codi que us proporcionem. D'aquesta manera tindreu un primer contacte amb els conceptes que hi exposem. Ara bé, **la lectura és només el principi!** Una vegada hàgiu llegit el contingut, no oblideu executar el codi proporcionat i modificar-lo per crear-ne variants que us permetin comprovar que heu entès la seva funcionalitat i explorar-ne els detalls d'implementació. Per últim, us recomanem també consultar la documentació enllaçada per explorar amb més profunditat les funcionalitats dels mòduls presentats.

Per guardar possibles modificacions que feu sobre aquest notebook, us aconsellem que munteu la unitat de Drive a Google Colaboratory (colab). Heu d'executar les instruccions següents:

```
[ ]: from google.colab import drive  
drive.mount('/content/drive')
```

```
[ ]: %cd /content/drive/MyDrive/Colab_Notebooks/prog_datasci_4
```

1.1.2 Introducció

A continuació es presentaran explicacions i exemples d'ús de la llibreria NumPy. Recordeu que podeu anar executant els exemples per obtenir-ne els resultats.

A continuació s'inclou la taula de continguts, que podeu fer servir per navegar pel document:

1 Introducció

2	Primers passos
3	Operacions amb matrius
4	Slicing i iteració
5	Exemple: el joc de la vida de Conway
6	Exercicis i preguntes teòriques
6.1	Instruccions importants
# 1	Introducció

`NumPy` va néixer amb el nom de `Numeric` i, poc a poc, va anar centralitzant molts altres desenvolupaments d'altres autors (com `NumArray`) per acabar sent la llibreria que avui dia coneixem amb el nom de `NumPy`. Actualment `NumPy` està integrada a la [SciPy stack](#), encara que continua sent un paquet independent de `SciPy` (més tard en parlarem).

`NumPy` és una llibreria de càlcul matricial en origen, tot i que avui dia implementa moltes altres funcionalitats interessants com són eines per integrar codi C/C i Fortran, funcions d'àlgebra lineal, transformacions de Fourier, etc.

2 2 Primers passos

Importarem la llibreria:

```
[62]: # A la línia següent, importem NumPy i li donem un nom més curt
      # perquè ens sigui més còmode fer les crides
      import numpy as np
```

A `NumPy`, l'objecte bàsic és una llista multidimensional de nombres (normalment) del mateix tipus.

```
[63]: # Exemple bàsic, un punt a l'espai:
      p = np.array([1, 2, 3])
```

A `NumPy`, a les dimensions se les coneix amb el nom d'eixos (*axes*), i al nombre d'eixos, rang (*rank*). *Array* és un àlies per referir-se al tipus d'objecte *numpy.ndarray*.

Algunes propietats importants dels *arrays* són les següents: * **`ndarray.ndim`**: el nombre d'eixos de l'objecte *array* (matriu). * **`ndarray.shape`**: una tupla de nombres enters indicant la longitud de les dimensions de la matriu. * **`ndarray.size`**: el nombre total d'elements de la matriu.

```
[64]: # Crearem una matriu bidimensional 3x2 (tres files, dues columnes).
      a = np.arange(3*2) # Creem un array unidimensional de sis elements
      print('Array unidimensional:')
      print(a)
      a = a.reshape(3,2) # Li donem "forma" de matriu 3x2.
      print('Matriu 3x2:')
      print(a)
```

```
Array unidimensional:  
[0 1 2 3 4 5]  
Matriu 3x2:  
[[0 1]  
 [2 3]  
 [4 5]]
```

```
[65]: # La dimensió és 2.  
a.ndim
```

```
[65]: 2
```

```
[66]: # Longitud de les dimensions.  
a.shape
```

```
[66]: (3, 2)
```

```
[67]: # El nombre total d'elements:  
a.size
```

```
[67]: 6
```

A l'hora de crear *arrays*, tenim diferents opcions:

```
[68]: # Creem un array (vector) de deu elements:  
z = np.zeros(10)  
print(z)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
[69]: # Podem canviar qualsevol dels valors d'aquest vector accedint a la seva posició:  
z[4] = 5.0  
z[-1] = 0.1  
print(z)
```

```
[0. 0. 0. 0. 5. 0. 0. 0. 0. 0.1]
```

```
[70]: # La funció 'arange' ens permet definir diferents opcions, com el punt d'inici i  
      ↪ el de final:  
a = np.arange(10,20)  
print(a)
```

```
[10 11 12 13 14 15 16 17 18 19]
```

```
[71]: # L'últim argument ens permet utilitzar un pas de 2:  
a = np.arange(10,20,2)  
print(a)
```

```
[10 12 14 16 18]
```

```
[72]: # Podem crear arrays des de llistes de Python de diverses dimensions:  
a = np.array([[1, 2, 3], [4, 5, 6]])  
print(a)
```

```
[[1 2 3]  
 [4 5 6]]
```

3 Operacions amb matrius

NumPy implementa totes les operacions habituals amb matrius.

```
[73]: A = np.array([[1,0], [0,1]])  
B = np.array([[1,2], [3,4]])
```

```
[74]: # Suma de matrius  
print(A+B)
```

```
[[2 2]  
 [3 5]]
```

```
[75]: # Resta de matrius  
print(A-B)
```

```
[[ 0 -2]  
 [-3 -3]]
```

```
[76]: # Multiplicació element per element  
print(A*B)
```

```
[[1 0]  
 [0 4]]
```

```
[77]: # Multiplicació de matrius  
print(A.dot(B))
```

```
[[1 2]  
 [3 4]]
```

```
[78]: # Potència  
print(B**2)
```

```
[[ 1  4]  
 [ 9 16]]
```

4 Slicing i iteració

Els *arrays* en NumPy suporten la tècnica de *slicing* de Python. Podem fer servir aquesta tècnica per recuperar valors d'un array:

```
[79]: # Definim un numpy array de 0 a 9.
a = np.arange(10)
print(a)

# Obtenim els elements des de la posició 2 fins la 4 de l'array (els índexs
↳ comencen a 0).
print(a[2:5])
```

```
[0 1 2 3 4 5 6 7 8 9]
[2 3 4]
```

```
[80]: # Tots els elements a partir de la segona posició:
a[2:]
```

```
[80]: array([2, 3, 4, 5, 6, 7, 8, 9])
```

```
[81]: # Podem iterar per cada element de l'array:
for i in a:
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

```
[82]: # Ara definim un array multidimensional.
A = np.arange(18).reshape(6,3)
print(A)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]
 [12 13 14]
 [15 16 17]]
```

```
[83]: # Accedim a una posició concreta de l'array multidimensional

# Primera alternativa:
print(A[3][2])
# Segona alternativa
```

```
print(A[3, 2])
```

11

11

```
[84]: # Accedim a una fila sencera (en aquest cas, la fila 2)
print(A[2])
```

[6 7 8]

```
[85]: # I ara a una columna sencera (en aquest cas, la columna 0)
print(A[:, 0])
```

[0 3 6 9 12 15]

```
[86]: # Obtenim totes les files fins a la cinquena
print(A[:5])
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]
 [12 13 14]]
```

```
[87]: # Podem iterar a través dels elements de l'array multidimensional
for i in range(A.shape[0]):
    for j in range(A.shape[1]):
        print("Element: {}".format(A[i, j]))
```

```
Element: 0
Element: 1
Element: 2
Element: 3
Element: 4
Element: 5
Element: 6
Element: 7
Element: 8
Element: 9
Element: 10
Element: 11
Element: 12
Element: 13
Element: 14
Element: 15
Element: 16
Element: 17
```

També podem fer servir *slicing* en assignacions:

```
[88]: # Modifiquem el valor de la posició 3, 2
A[3, 2] = 42
print(A)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 42]
 [12 13 14]
 [15 16 17]]
```

```
[89]: # Modifiquem el valor d'una fila completa
A[2] = [101, 101, 101]
print(A)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [101 101 101]
 [ 9 10 42]
 [12 13 14]
 [15 16 17]]
```

```
[90]: # Igualem tots els elements a 0 (l'operador ... afegeix tots els : necessaris).
A[...] = 0
print(A)
```

```
[[0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]]
```

5 Exemple: el joc de la vida de Conway

El joc de la vida és un exemple clàssic d'autòmat cel · lular creat el 1970 pel famós matemàtic John H. Conway. Al problema clàssic, es representen en una matriu bidimensional cèl · lules que viuran o moriran depenent del nombre de veïns en un determinat pas de la simulació. Cada cèl · lula té vuit veïns (les caselles adjacents en un tauler bidimensional) i pot estar viva (1) o morta (0). Les regles clàssiques entre transició vida/mort són les següents:

- Una cèl · lula morta amb exactament tres cèl · lules veïnes vives al torn següent estarà viva.
- Una cèl · lula viva amb dues o tres cèl · lules veïnes vives segueix viva, en un altre cas mor o roman morta (solitud en cas d'un número més petit que 2, superpoblació si és major a 3).

Aquest problema (o joc) té moltes variants depenent de les condicions inicials, si el tauler (o món) té vores o no, o si bé les regles de vida o mort són alterades.

A continuació teniu un exemple del joc clàssic implementat a NumPy:

```
[24]: # Autor: Nicolas Rougier
# Font: http://www.labri.fr/perso/nrougier/teaching/numpy.100/

SIZE = 10
STEPS = 10

def iterate(Z):
    # Comptem veïns:
    N = (Z[0:-2,0:-2] + Z[0:-2,1:-1] + Z[0:-2,2:] +
         Z[1:-1,0:-2] + Z[1:-1,2:] +
         Z[2:,0:-2] + Z[2:,1:-1] + Z[2:,2:])

    # Apliquem les regles:
    birth = (N==3) & (Z[1:-1,1:-1]==0)
    survive = ((N==2) | (N==3)) & (Z[1:-1,1:-1]==1)
    Z[...] = 0
    Z[1:-1,1:-1][birth | survive] = 1
    return Z

# Creem un tauler amb cèl·lules vives o mortes de manera aleatòria.
Z = np.random.randint(0,2,(SIZE, SIZE))
# Simulem durant els passos indicats:
for i in range(STEPS):
    Z = iterate(Z)
# Mostrem el tauler en el pas final:
print(Z)
```

```
[[0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 1 1 0 0]
 [0 0 0 0 0 0 0 1 1 0]
 [0 0 0 0 1 1 0 0 0 0]
 [0 0 1 1 1 0 0 0 0 0]
 [0 0 0 1 1 0 0 0 0 0]
 [0 0 1 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]]
```

6 Exercicis i preguntes teòriques

La part avaluable d'aquesta unitat consisteix en el lliurament d'un fitxer IPython Notebook amb extensió IPYNB que contindrà els diferents exercicis i les preguntes teòriques que s'han de contestar. Trobareu el fitxer (`prog_datasci_4_scilib_entrega.ipynb`) amb les activitats a la mateixa carpeta que aquest notebook que esteu llegint. **Hi ha un sol fitxer d'activitats per a tota la unitat, que cobreix totes les llibreries que s'hi treballen.**

6.1 Instruccions importants

És molt important que a l'hora de lliurar el fitxer Notebook amb les vostres activitats us assegureu

que:

1. Les vostres solucions siguin originals. Esperem no detectar-hi còpia directa entre estudiants.
2. Tot el codi estigui correctament documentat. El codi sense documentar equivaldrà a un 0.
3. El fitxer comprimit que lliureu és correcte (conté les activitats de la PAC que heu de lliurar).

Per fer el lliurament, heu d'anar a la carpeta del drive **Colab Notebooks**, clicant botó dret a la PAC en qüestió i fent **Download**. D'aquesta manera, us baixareu la carpeta de la PAC comprimida en **zip**. Aquest és l'arxiu que heu de pujar al campus virtual de l'assignatura. ‘

3 Autors

- Autor original **Brian Jiménez Garcia**, 2016.
- Actualitzat per **Cristina Pérez Solà**, 2017 i 2019.

<div style="width:0%;"> </div>

</div>